

CodeLeWM: A Reproducible Code World-Model Study with Negative Action-Use Results and a Narrow Downstream Reranking Slice

Abdelhamid Bakhta
StarkWare

<https://github.com/AbdelStark/CodeLeWM>

Draft of June 8, 2026

Abstract

Joint-embedding predictive architectures learn by predicting future latent states rather than reconstructing observations. CodeLeWM ports this recipe to Python code transitions and asks whether a compact latent world model can add value when ranking candidate code changes. The answer is mixed. The original commit-edit substrate and the v0.2 action-swap intervention are negative: text-action retrieval reaches Recall@1 0.263 and MRR 0.370, while the no-action baseline reaches Recall@1 0.441 and MRR 0.533. Execution-trace substrates repair collapse and produce healthy diagnostics: v0.9 retrieval beats all retrieval controls, mutation surprise AUC is 1.0 on both seeds, and the downstream HumanEval WS-D replay reaches CodeLeWM pass@1 0.9787 on both seeds, beating no-action by +10.64 and +8.51 points with confidence intervals excluding zero. The aggregate public claim remains closed. MBPP-Plus is saturated with CodeLeWM, no-action, and lexical all at pass@1 1.0000; broad semantic surprise has no scorable semantic pairs; representation probes do not beat controls; and the final score rows do not expose a standalone `p_pass` score key. We therefore present CodeLeWM as a reproducible negative and diagnostic study with a narrow HumanEval WS-D positive slice, not as evidence that the model generally improves coding.

1 Introduction

World models are attractive because they promise useful predictions without directly imitating every observation. In joint-embedding predictive architectures (JEPAs), the model predicts the latent of a future state rather than reconstructing pixels or tokens [2, 6]. LeWorldModel applies the idea to action-conditioned visual dynamics from pixels [15]. CodeLeWM asks whether the same latent-transition framing can be made useful for code.

The intended downstream object is not a code generator. It is a scorer and reranker. Given a before-state, an edit instruction or execution input, and a set of candidate after-states, CodeLeWM estimates which candidate best matches the learned transition. The transition contract is:

$$\hat{z}_{\text{after}} = P(E(s_{\text{before}}), A(a)). \quad (1)$$

This paper reports the full evidence arc rather than only the strongest slice. The early commit-message-conditioned edit substrate failed the intended action-use gate. A later execution-substrate pivot produced non-collapsed checkpoints and useful diagnostics. The final v0.9/v1.0 downstream replay then found a narrow HumanEval WS-D win but did not clear the aggregate downstream

claim because MBPP-Plus was saturated by controls. This is an honest end state for the artifact: reproducible systems evidence, negative model-quality evidence on action use, one narrow positive reranking slice, and explicit blocked claims.

Contributions.

1. A manifest-backed code world-model harness for dataset construction, training, retrieval, surprise, latent probes, downstream reranking, and paper-demo replay.
2. A negative action-use result on commit-edit transitions, including v0.2 no-action dominance and unsupported latent semantic structure.
3. A diagnostic execution-substrate result showing non-collapse, retrieval lift over controls, and mutation-surprise separation.
4. A final downstream replay showing HumanEval WS-D lift over no-action on two seeds while closing the aggregate claim on MBPP-Plus saturation and broader representation/semantic gates.

2 Related Work

Joint-embedding prediction. I-JEPA and V-JEPA learn feature predictions without pixel reconstruction [2, 6]. LeWorldModel studies stable end-to-end JEPA-style world modeling from pixels [15]. Anti-collapse regularization is central to this family, with related ideas in VICReg, Barlow Twins, and LeJEPA [4, 5, 19].

Code modeling and execution signals. Pretrained code models such as CodeBERT, GraphCodeBERT, UniXcoder, and CodeT5 learn program representations or generation models from code and natural language [10–12, 17]. HumanEval, MBPP, APPS, CodeNet, and EvalPlus-style benchmark extensions provide candidate tasks and labels for coding evaluation [3, 8, 13, 14, 16]. Generated tests have also been used for reranking code generations [7]. Execution-aware pre-training and execution-trace work provide a complementary signal for program behavior [1, 9, 18]. CodeLeWM differs by treating code changes and execution records as manifested latent transitions and by refusing to execute untrusted candidate code during scoring.

3 Model and Artifact Contract

CodeLeWM uses bounded code or execution states, an action encoder, a transition predictor, and anti-collapse regularization. The exact module and objective variants changed across research iterations, but the public contract did not: every publishable run writes schema-versioned reports, artifact manifests, checksums, and secret-scan evidence. Candidate code and generated patches are untrusted. They may be parsed or scored as text, but they are not imported, executed, or test-run by the model-scoring path.

The final paper-demo contract is deterministic. It replays checked-in v0.9 WS-D completion labels and score rows for four slices: seed 42 and seed 1729 crossed with HumanEval WS-D and MBPP-Plus WS-D. Replay mode records `score_source=replay_existing_scores`; it is not a fresh checkpoint scoring run and not a live OpenRouter demo. The committed artifact id is `demo_report-e6fc06c328eed245`.

Table 1: Evidence arc and public claim status.

Milestone	Main evidence	Open claim	Closed or blocked claim
v0.2	action-swap A10G run	systems pipeline	action-use, latent axes, downstream utility
v0.6	execution substrate	diagnostics	downstream HumanEval/MBPP utility
v0.8	execution repair	HumanEval WS-D slice	MBPP, magnitude probe, broad semantic gate
v0.9	two-seed final gate suite	HumanEval WS-D slice	MBPP, representation, broad semantic gate
v1.0	paper-demo replay	reproducible package	demo aggregate downstream usefulness

4 Evidence Arc

Table 1 summarizes the evidence by milestone. The final claim audit is `docs/benchmark/V1_0_FINAL_CLAIM_AUDIT_2026-06-08.md`.

4.1 Negative action-use evidence

The v0.2 action-swap/inverse-action run directly tested whether action conditioning beat no-action on held-out retrieval. It did not. Text-action Recall@1 was 0.263 and MRR was 0.370, while no-action reached Recall@1 0.441 and MRR 0.533. Exact-same-before, near-before, same-file, action-cluster, edit-shape, and random action-contrast pools also failed to show the required margin over no-action. Latent probes were explicitly unsupported for semantic structure: lexical or metadata controls beat the latent views on available targets.

4.2 Execution-substrate diagnostics

The execution substrate changed the state/action/next-state signal from noisy commit messages to deterministic execution triples. This improved training health and diagnostics. By v0.9, retrieval lift over controls passed on both seeds: Recall@1 was 0.2538 and 0.2731, while all listed retrieval controls were below 0.024 Recall@1. Mutation surprise AUC was 1.0 on both seeds. These are training-health and diagnostic results; they do not by themselves prove downstream coding usefulness.

Representation claims remain closed. The v0.9 **passed** probe reached 0.6552 and 0.6158 for $\hat{z}_{\text{after}}$, but no-action or before-state controls were stronger. The `output_magnitude_bucket` target was evaluable in v0.9 but tied a direct `z_after` control. Broad semantic surprise also remains closed because the semantic-decoy categories have zero scorable aligned pairs in the final pack.

5 Final Downstream Replay

The final paper-demo replay is the paper’s central downstream table. It is derived from `docs/benchmark/v1_0/paper_demo/reports/paper_demo_report.json` and summarized in Table 2.

HumanEval WS-D is the narrow positive slice: CodeLeWM reaches pass@1 0.9787 on both seeds and beats no-action by +10.64 and +8.51 points. The 95% bootstrap confidence intervals for lift over no-action are [2.13, 21.28] and [2.13, 17.02], respectively. LLM-order is much weaker at pass@1 0.1489.

Table 2: Final v1.0 paper-demo replay. The aggregate claim gate is closed even though the HumanEval WS-D slices are open.

Seed	Benchmark	CodeLeWM	No-action	LLM-order	Lexical	Lift	Gate
42	HumanEval WS-D	0.9787	0.8723	0.1489	0.6596	+10.64	open
42	MBPP-Plus WS-D	1.0000	1.0000	0.1765	1.0000	+0.00	closed
1729	HumanEval WS-D	0.9787	0.8936	0.1489	0.6596	+8.51	open
1729	MBPP-Plus WS-D	1.0000	1.0000	0.1765	1.0000	+0.00	closed

MBPP-Plus is the counterweight. CodeLeWM reaches pass@1 1.0000, but so do no-action and lexical controls. The measured lift over no-action is exactly +0.00 points on both seeds with confidence interval [0.00, 0.00]. This makes the aggregate downstream claim closed. The evidence supports a narrow HumanEval WS-D result, not a claim that CodeLeWM generally improves patch ranking.

6 Why No-Action and Generalization Remain Hard

The no-action baseline is strong because many code transitions are sparse: large portions of the after-state are identical to the before-state, and many candidate completions can be ranked using surface similarity. A world model must learn when the action changes behavior, not merely when text resembles the prompt. The v0.2 failure shows that a commit message is often too weak or too underspecified to force this distinction.

The MBPP-Plus result exposes a different problem. The final WS-D MBPP-Plus pack is saturated. CodeLeWM, no-action, and lexical all choose passing candidates at rank 1 on both seeds. Saturation prevents a lift claim because the model did not add measurable value beyond simple controls. This is not a bug in the paper demo; it is the reason the aggregate claim gate must close. A future positive claim needs harder, sufficiently large, public-safe downstream labels where no-action and lexical controls do not already solve the slice.

The representation and semantic-surprise blockers reinforce the same point. Healthy retrieval and mutation diagnostics show that the model is not collapsed and can separate some generated decoys. They do not show stable semantic axes or broad behavioral understanding. Those claims require probes and decoy packs that beat controls across seeds and have enough scorable coverage.

7 Claim Boundary

Table 3 is the paper-level claim audit. It mirrors `docs/papers/codelewm_final_claim_audit.md`.

8 Reproducibility

The source repository is <https://github.com/AbdelStark/CodeLeWM>. The main artifact reports used in this paper are:

- `docs/benchmark/V0_2_ACTION_SWAP_HF_RESULTS_2026-05-20.md`
- `docs/benchmark/EXECUTION_V0_8_RESULTS_2026-06-05.md`
- `docs/benchmark/EXECUTION_V0_9_RESULTS_2026-06-07.md`

Table 3: Allowed and blocked public claims.

Claim	Verdict	Evidence
Reproducible artifact pipeline	allowed	manifests, scans, HF artifacts, paper demo
HumanEval WS-D reranking slice	allowed	0.9787 pass@1 on both seeds
Broad coding improvement	blocked	aggregate downstream gate closed
MBPP-Plus/generalization	blocked	zero lift over no-action and lexical
Action-use model-quality	blocked	v0.2 no-action dominance
Semantic latent axes	blocked	probes do not beat controls
Standalone p_pass down-stream score	not recorded	final rows mark p_pass missing

- docs/benchmark/PAPER_DEMO_V1_0_ARTIFACTS_2026-06-08.md
- docs/benchmark/V1_0_FINAL_CLAIM_AUDIT_2026-06-08.md
- docs/benchmark/v1_0/paper_demo/manifest.json

The deterministic replay command for the final demo is:

```
uv run scripts/paper-demo --out docs/benchmark/v1_0/paper_demo --overwrite --json
```

The paper build command is:

```
scripts/build-codelewm-final-paper
```

9 Limitations

- The final positive result is a slice result, not a broad benchmark result.
- The final paper demo replays checked-in v0.9 score rows. It is reproducible and manifest-backed, but it is not a fresh checkpoint scoring run.
- MBPP-Plus saturation prevents measuring lift on that slice.
- The representation probes do not support naming semantic dimensions.
- Broad semantic surprise remains blocked by scorable-pair coverage.
- Candidate code remains untrusted; the model-scoring path does not execute candidate patches.
- The results do not compare against large pretrained code models as generators. CodeLeWM is a scorer/reranker artifact.

10 Conclusion

CodeLeWM is a completed reproducible research artifact, not a broad coding assistant. The artifact discipline worked: data, checkpoints, reports, claim gates, manifests, and scans can be regenerated and inspected. The original action-use hypothesis did not work: no-action stayed stronger on commit-edit retrieval. The execution substrate gave healthier model diagnostics and a narrow

HumanEval WS-D downstream win, but MBPP-Plus saturation and representation blockers keep the public model-quality claim closed. The most useful conclusion is methodological: code world-model claims should be published with their baselines and failure gates intact, especially when no-action and lexical controls are strong.

References

- [1] Jordi Armengol-Estap, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael F. P. O’Boyle, Sida Wang, Gabriel Synnaeve, and Hugh James Leather. What i cannot execute, i do not understand: Training and evaluating llms on program execution traces, 2025. URL <https://arxiv.org/abs/2503.05703>.
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL <https://arxiv.org/abs/2301.08243>.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [4] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL <https://arxiv.org/abs/2511.08544>.
- [5] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022. URL <https://arxiv.org/abs/2105.04906>.
- [6] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL <https://arxiv.org/abs/2404.08471>.
- [7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022. URL <https://arxiv.org/abs/2207.10397>.
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [9] Yangruibo Ding, Ben Steenhoeck, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. Traced: Execution-aware pre-training for source code, 2023. URL <https://arxiv.org/abs/2306.07487>.

- [10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages, 2020. URL <https://arxiv.org/abs/2002.08155>.
- [11] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. Graphcodebert: Pre-training code representations with data flow, 2021. URL <https://arxiv.org/abs/2009.08366>.
- [12] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation, 2022. URL <https://arxiv.org/abs/2203.03850>.
- [13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps, 2021. URL <https://arxiv.org/abs/2105.09938>.
- [14] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. URL <https://arxiv.org/abs/2305.01210>.
- [15] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. URL <https://arxiv.org/abs/2603.19312>.
- [16] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks, 2021. URL <https://arxiv.org/abs/2105.12655>.
- [17] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021. URL <https://arxiv.org/abs/2109.00859>.
- [18] Wojciech Zaremba and Ilya Sutskever. Learning to execute, 2015. URL <https://arxiv.org/abs/1410.4615>.
- [19] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction, 2021. URL <https://arxiv.org/abs/2103.03230>.